

SE4030 â Secure Software Development Assignment Report

Car Booking Web Application Hardening

Team Members: <Member Name 1> (<Index Number>), <Member Name 2> (<Index Number>), <Member Name 3> (<Index Number>), <Member Name 4> (<Index Number>)

Submission Date: <insert date>

1. Overview

We assessed the legacy PHP car booking portal to locate high-impact security weaknesses, implemented remediations, and added Google OAuth 2.0 based authentication. The hardened codebase is now stored in the secured repository referenced in the README file.

2. Assessment Methodology

- Manual secure code review of PHP pages and includes with a focus on authentication, booking flows, and administrative tooling.
- Static analysis using built-in PHP linting (php -l) across the modified scripts to prevent syntax and parse errors.
- Targeted functional testing of registration, login, booking, and logout scenarios over both local accounts and Google sign-in.

3. Vulnerabilities Identified and Remediated

V1 â Weak Session Cookie Security: The original application relied on default PHP session handling without HttpOnly, Secure, or SameSite flags, allowing theft or reuse of session identifiers. We centralised session bootstrap logic in inc/security.php to set hardened cookie parameters and regenerate IDs after login.

V2 â Absence of CSRF Protection: Login, registration, and booking forms lacked anti-CSRF checks, enabling forged requests. Each form now embeds a per-session token generated by generateCsrfToken() and validates it server-side before mutating state.

V3 â SQL Injection in Authentication Flows: Raw string interpolation was used when looking up users, making login bypass trivial. All database access has been refactored to use parameterised mysqli prepared statements across admin/index.php, login.php, register.php, and carDetails.php.

V4 â Insecure Password Storage: User and admin passwords were stored as legacy MD5 hashes. Registration now stores passwords using password_hash(), and the login flows transparently upgrade any remaining MD5 hashes to bcrypt on successful authentication.

V5 â Stored and Reflected XSS Exposure: User-provided values such as car booking notes and vehicle metadata were output without escaping, enabling script injection. The new escape() helper encodes all dynamic output in templates, and booking messages are stripped of HTML and length-limited before storage.

V6 â Booking Input Validation Failures: Date fields accepted arbitrary strings, which could corrupt records and bypass availability checks. Server-side validation now normalises dates, rejects invalid ranges, and caps free-form messages to 500 characters before executing the INSERT statement.

V7 â Hardcoded Database Credentials and Silent Failures: Connection scripts contained plaintext credentials with suppressed errors, risking credential leakage and unstable deployments. The connection bootstrap now sources credentials from environment variables, enables mysqli exceptions, and fails fast with a sanitised error response.

4. OAuth 2.0 Integration

We introduced a Google OAuth 2.0 login path that complements local accounts. oauth/google-login.php initiates the authorisation request, while oauth/google-callback.php validates the returned state, exchanges the code for tokens, fetches the OpenID Connect userinfo payload, and either links or provisions the user record inside the existing users table. OAuth-sourced accounts store provider metadata and bypass password storage, and session regeneration occurs before redirecting the user to their dashboard.

5. Testing and Validation

- php -l executed against all touched PHP scripts to ensure they parse correctly after each change.
- Manual regression testing covering: new registrations, legacy MD5 account login upgrade, failed login handling, booking creation with valid and invalid dates, and Google sign-in happy path.

6. Remaining Known Risks

- Rate limiting and account lockout were not implemented; brute-force resistance should be added in a future iteration.
- Transport security (HTTPS enforcement and HSTS) must be configured at the web server or load balancer level when deploying.

7. Secure Development Best Practices

- Adopt peer code reviews focused on security-sensitive areas before merging changes.

- Maintain automated dependency and vulnerability scanning as part of the CI pipeline.
- Document secure coding patternsâ prepared statements, output encoding, and CSRF defencesâ for future contributors.

Appendix â Environment Configuration

Set the following environment variables before running the application in production:

DB_HOST, DB_USER, DB_PASSWORD, DB_NAME, GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET, GOOGLE_REDIRECT_URI. Default development fallbacks exist for the database values, but OAuth credentials must always be provided.